Geração de Testes Unitários Jokenpo Java

https://gemini.google.com/app/0ee14c72b2326c12

User prompt: Você é um testador experimente de testes unitários. Gere casos de testes para a classe Java abaixo em um único arquivo Java com imports usando JUnit 5 e Java 8. Use somente Junit para os testes unitários

Response: Aqui está a classe de testes unitários para o seu jogo de Jokenpo.

Como um bom testador, minha estratégia foi cobrir todos os cenários possíveis (caminho feliz e casos de exceção/limite) para garantir que a lógica do jogo esteja blindada contra bugs. Dividi os testes em quatro blocos lógicos: empates, vitórias do jogador 1, vitórias do jogador 2 e entradas inválidas.

Utilizei o **JUnit 5** e a estrutura do **Java 8**, instanciando a classe testada em um método com a anotação @BeforeEach para manter o código limpo e evitar repetições.

Código de Teste (JokempoTest.java)

```
package jokempo;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

public class JokempoTest {

    private Jokempo jokempo;

    @BeforeEach
    public void setUp() {
        // Inicializa a classe antes de cada método de teste
        jokempo = new Jokempo();
    }

    // --- TESTES DE EMPATE (Retorno 0) ---

    @Test
    public void deveRetornarEmpateQuandoAmbosEscolhemPapel() {
        assertEquals(0, jokempo.jogar(1, 1), "Deveria dar empate (0) quando
ambos escolhem Papel (1)");
    }
```

```
    @Test
    public void deveRetornarEmpateQuandoAmbosEscolhemPedra() {
        assertEquals(0, jokempo.jogar(2, 2), "Deveria dar empate (0) quando
ambos escolhem Pedra (2)");
    }

    @Test
    public void deveRetornarEmpateQuandoAmbosEscolhemTesoura() {
        assertEquals(0, jokempo.jogar(3, 3), "Deveria dar empate (0) quando
ambos escolhem Tesoura (3)");
    }

    // --- TESTES DE VITÓRIA DO JOGADOR 1 (Retorno 1) ---

    @Test
    public void deveRetornarVitoriaJogador1QuandoPapelGanhaDePedra() {
        assertEquals(1, jokempo.jogar(1, 2), "Jogador 1 (Papel) deveria ganhar
do Jogador 2 (Pedra)");
    }

    @Test
    public void deveRetornarVitoriaJogador1QuandoPedraGanhaDeTesoura() {
        assertEquals(1, jokempo.jogar(2, 3), "Jogador 1 (Pedra) deveria ganhar
do Jogador 2 (Tesoura)");
    }

    @Test
    public void deveRetornarVitoriaJogador1QuandoTesouraGanhaDePapel() {
        assertEquals(1, jokempo.jogar(3, 1), "Jogador 1 (Tesoura) deveria
ganhar do Jogador 2 (Papel)");
    }

    // --- TESTES DE VITÓRIA DO JOGADOR 2 (Retorno 2) ---

    @Test
    public void deveRetornarVitoriaJogador2QuandoPedraPerdeParaPapel() {
        assertEquals(2, jokempo.jogar(2, 1), "Jogador 2 (Papel) deveria ganhar
do Jogador 1 (Pedra)");
    }

    @Test
    public void deveRetornarVitoriaJogador2QuandoTesouraPerdeParaPedra() {
        assertEquals(2, jokempo.jogar(3, 2), "Jogador 2 (Pedra) deveria ganhar
do Jogador 1 (Tesoura)");
    }

    @Test
    public void deveRetornarVitoriaJogador2QuandoPapelPerdeParaTesoura() {
        assertEquals(2, jokempo.jogar(1, 3), "Jogador 2 (Tesoura) deveria
ganhar do Jogador 1 (Papel)");
    }
```

```
    // --- TESTES DE ENTRADAS INVÁLIDAS (Retorno -1) ---

    @Test
    public void deveRetornarInvalidoQuandoJogador1EnviaOpcaoAbaixoDoLimite() {
        assertEquals(-1, jokempo.jogar(0, 1), "Deveria retornar -1 quando
Jogador 1 for menor que 1");
        assertEquals(-1, jokempo.jogar(-5, 2), "Deveria retornar -1 para
valores negativos no Jogador 1");
    }

    @Test
    public void deveRetornarInvalidoQuandoJogador1EnviaOpcaoAcimaDoLimite() {
        assertEquals(-1, jokempo.jogar(4, 2), "Deveria retornar -1 quando
Jogador 1 for maior que 3");
    }

    @Test
    public void deveRetornarInvalidoQuandoJogador2EnviaOpcaoAbaixoDoLimite() {
        assertEquals(-1, jokempo.jogar(1, 0), "Deveria retornar -1 quando
Jogador 2 for menor que 1");
        assertEquals(-1, jokempo.jogar(2, -10), "Deveria retornar -1 para
valores negativos no Jogador 2");
    }

    @Test
    public void deveRetornarInvalidoQuandoJogador2EnviaOpcaoAcimaDoLimite() {
        assertEquals(-1, jokempo.jogar(3, 4), "Deveria retornar -1 quando
Jogador 2 for maior que 3");
    }

    @Test
    public void deveRetornarInvalidoQuandoAmbosJogadoresEnviamOpcoesInvalidas()
{
        assertEquals(-1, jokempo.jogar(0, 5), "Deveria retornar -1 quando ambos
jogadores enviam valores fora de 1 a 3");
    }
}
```

Detalhes Importantes:

- **Imports corretos:** Certifique-se de que sua IDE está configurada para reconhecer a biblioteca do JUnit 5 (org.junit.jupiter.api).
- **Mensagens Assertivas:** O terceiro parâmetro do assertEquals em JUnit 5 permite incluir uma mensagem caso o teste falhe, o que é uma boa prática em QA para ajudar a identificar rapidamente qual o problema no log.
